

jordanvalnet/code-dev-intel.ts

code-dev-intel : une chaîne de vérification de niveau industriel sur trois systèmes, mais quatre ensembles de tests, aucun bug étiqueté et une version de Node annoncée que rien n'exécute

Self-hosted AI code intelligence for TypeScript — symbol resolution, structural search, duplicate detection, and dependency graphs via MCP server.

4

FICHIERS DE TEST

vitest

4

WORKFLOWS CI

1

ISSUES OUVERTES

<https://github.com/jordanvalnet/code-dev-intel.ts> Licence AGPL-3.0 Onboard · profil qa

☑ Que fait ce projet et quels sont ses parcours critiques ?

VERDICT

Quatre services, trois surfaces d'entrée et un état persistant hors du dépôt : le parcours à qualifier en premier est la réutilisation du graphe entre deux exécutions, seul endroit où une réponse peut être fausse sans que rien ne plante.

`code-dev-intel.ts` est un serveur auto-hébergé d'intelligence de code TypeScript : il répond aux questions de navigation d'un agent IA — définitions, références, implémentations, plan d'un fichier, graphe d'imports, rayon d'impact d'un changement, recherche structurelle et détection de duplication — sans que le modèle relise le dépôt entier.

Il s'installe en dépendance de développement et se démarre par une commande unique, conçue pour être rejouée sans effet de bord : elle ne lance le service que s'il ne tourne pas, attend qu'il soit sain, et sort en succès s'il tournait déjà.

Trois surfaces d'entrée à qualifier séparément, car elles ne partagent que le cœur : MCP sur stdio, MCP en JSON-RPC sur `POST /mcp`, et les points HTTP `/tools/*` avec `/health` et `/tools/describe`.

Les parcours critiques, par ordre de dégâts en cas de défaut :

- 1 **Démarrage et santé** — `ensure` doit être idempotent : deux appels concurrents, un port déjà pris, un serveur déjà sain.
- 2 **Réponse sémantique** — `findReferences`, `findDefinitions`, `findImplementations` passent par le vérificateur de types : un faux positif de recherche texte ici est un bug fonctionnel, pas un détail de format.
- 3 **Graphe d'imports et rayon d'impact** — `dependencyGraph` et `impactedFiles` partagent le même résolveur et ne doivent jamais se contredire sur un même dépôt ; ce qu'ils n'ont pas su suivre doit ressortir dans `unresolved`, jamais être tu.
- 4 **Réutilisation entre deux exécutions** — le graphe est réécrit dans le répertoire de cache de l'utilisateur, hors du dépôt, et relu au démarrage suivant ; c'est le seul état qui survit au processus, donc le premier suspect d'un résultat périmé.
- 5 **Portabilité** — binaires natifs (recherche texte et structurelle) résolus par plateforme, gestion des chemins et de la casse propres à Windows et macOS.

☑ Comment est-il testé aujourd'hui ?

VERDICT

Quatre ensembles unitaires seulement, mais quatre niveaux de recette au-dessus : ici la qualité est portée par la chaîne de vérification, pas par les tests unitaires.

- ☐ `.github/` workflows GitHub Actions
- ☐ `__tests__/` tests unitaires vitest
- ☐ `docker/` docker-compose.yml et profils
- ☐ `docs/` documentation, dont docs/ai
- ☐ `perf/` benchmarks de performance
- ☐ `schemas/` schémas livrés avec le paquet
- ☐ `scripts/` bootstrap, smoke tests, benchmark
- ☐ `security/` ressources sécurité
- ☐ `services/` serveur MCP, indexeur, gouvernance, sécurité
- ☐ `.dockerignore` exclusions du build Docker
- ☐ `.gitignore` fichiers ignorés par git
- ☐ `.npmignore` exclusions du paquet npm
- ☐ `CHANGELOG.md` journal des versions
- ☐ `CONTRIBUTING.md` guide de contribution
- ☐ `LICENSE` licence AGPL-3.0-only
- ☐ `README.md` présentation et documentation principale
- ☐ ... et 6 autres entrées à la racine du dépôt

Tests unitaires : `vitest` , quatre ensembles relevés sous `__tests__/unit` — serveur MCP, indexeur, gouvernance, sécurité — lancés par `pnpm test` , configuration à la racine dans `vitest.config.ts` .

Ces quatre ensembles répondent aux quatre services du dépôt, mais un dossier de tests n'est pas une mesure : rien dans le relevé ne dit combien de cas ils contiennent.

Les niveaux au-dessus de l'unitaire ne sont pas dans `__tests__` : ils sont dans `scripts/` , joués par la chaîne de vérification.

NIVEAU	OÙ	COMMANDE	CE QU'IL PROUVE
Unitaire	<code>__tests__/unit</code> (4 ensembles)	<code>pnpm test</code>	Le comportement des services, sur les systèmes qui savent lancer vitest
Auto-test du binaire compilé	<code>dist/</code> , <code>job</code> <code>quality</code> en Node 20	<code>node ./dist/code-intel-mcp/src/server.js --self-test</code>	Le paquet compilé démarre, trouve ses binaires natifs et répond à un appel d'outil
Recette de livraison	<code>scripts/release-smoke.mjs</code>	<code>archive pnpm pack</code> installée dans un projet jetable	Ce qu'un utilisateur reçoit vraiment : fichiers manquants, dépendance optionnelle absente, chemin Windows
Recette de l'indexeur	<code>scripts/indexer-smoke.mjs</code>	<code>pnpm ci:indexer</code>	L'indexeur en modes git-diff et impacted
Sécurité	<code>services/security/opengrep-runner.ts</code>	<code>pnpm ci:security</code>	Scan statique OpenGrep, rapport SARIF publié en artefact

NIVEAU	OÙ	COMMANDE	CE QU'IL PROUVE
Performance	perf/	budget par système	Des seuils de temps, dans un workflow séparé

La couverture est mesurée une seule fois, sur ubuntu en Node 24, et publiée en texte et en résumé JSON — mais le chiffre lui-même n'est pas relevé ici. Donnée non relevée : le taux de couverture ; à lire dans le journal d'exécution le plus récent, <https://github.com/jordanvalnet/code-dev-intel.ts/actions>¹.

☑ Que dit la CI et quand tourne-t-elle ?

VERDICT

Cinq combinaisons système/Node à chaque push, recette de l'archive publiée sur trois systèmes : la porte est solide, mais la version de Node annoncée aux utilisateurs, la 18, ne passe par aucune porte.

Quatre workflows sont déclarés dans `.github/workflows` ; le principal, `ci.yml`, tourne à chaque push sur toute branche et à chaque pull request — donc rien n'attend une demande de fusion pour être vérifié.

`ci.yml` enchaîne quatre travaux, les trois derniers conditionnés à la réussite du premier : `quality`, puis `release-smoke`, `indexer-smoke` et `security`.

`quality` est une matrice de cinq combinaisons, sans arrêt au premier échec — un système qui casse ne masque donc pas les autres :

SYSTÈME	NODE	SUITE	CE QUI TOURNE
ubuntu	20	<code>runtime-compat</code>	lint, type-check, build, auto-test du compilé — pas de vitest
ubuntu	22	<code>full</code>	lint, type-check, tests
ubuntu	24	<code>full</code>	lint, type-check, tests, couverture
windows	24	<code>full</code>	lint, type-check, tests
macos	24	<code>full</code>	lint, type-check, tests

`release-smoke` rejoue les trois systèmes sur l'archive de publication : c'est le filet le plus proche de l'utilisateur final.

Trou de couverture n° 1, à porter au plan de test : le manifeste annonce Node 18 ou plus, alors que l'outillage de développement exige Node 22 (`vitest 5`, `--experimental-strip-types`) ; Node 20 ne voit passer que le binaire compilé, et **Node 18 n'est exécuté nulle part**.

Les déclencheurs des trois autres workflows — `perf-budget`, `pr-memory-reference`, `security` — ne sont pas relevés ; à vérifier sur la page des workflows du dépôt².

☑ Où sont les bugs connus ?

VERDICT

Zéro bug connu ne veut pas dire zéro bug : aucune étiquette n'existe dans ce dépôt, donc la file d'issues ne peut rien mesurer.

Une seule demande ouverte, aucune fermée sur les trente derniers jours, et **aucune étiquette posée sur quoi que ce soit** : il n'existe donc pas de bug déclaré comme tel dans ce dépôt.

La demande ouverte est l'issue #1, « Anti-Amnesia & Workflow Hardening for code-dev-intel.ts », sans aucun commentaire : un chantier de durcissement du processus, pas un défaut constaté.

Aucune issue n'est marquée pour un premier contributeur, donc aucune porte d'entrée balisée pour qui voudrait reproduire un défaut.

À lire comme un manque d'instrumentation, pas comme une preuve d'absence de défauts : sans étiquette `bug`, il n'y a ni compteur, ni tendance, ni tri possible.

En attendant, les régressions corrigées se lisent dans le journal des versions, qui décrit les corrections version par version — c'est aujourd'hui la seule trace exploitable d'un défaut passé.

Première recommandation opérationnelle : ouvrir les étiquettes `bug`, `regression` et `blocked` avant de commencer une campagne, sinon les défauts trouvés seront irrécupérables dans la file³.

☑ Comment reproduire et signaler un bug ici ?

VERDICT

Reproduire tient en trois commandes, mais un rapport n'est recevable qu'avec ses preuves : commandes, sorties, et le test refait sans réutilisation du graphe.

La file de changements est vide et le rythme est court : aucune pull request ouverte, une seule fusionnée sur trente jours — #3, « ci: three-OS quality matrix, multi-OS release smoke, per-OS perf budget », fusionnée le 6 septembre 2026.

Reproduire tient en trois commandes, sur un dépôt TypeScript de test — la dernière est le plus petit cas reproductible transmissible tel quel :

```
pnpm add -D code-dev-intel.ts
pnpm exec code-dev-intel ensure --workspaceRoot=. --port=4545
curl -X POST http://127.0.0.1:4545/tools/findReferences -H "Content-Type: application/json" -d '{"workspaceRoot":"/chemin/absolu","filePath":"src/feature/use-case.
```

Avant de conclure à un défaut de résolution, refaire le test sans réutilisation du graphe (`CODE_INTEL_GRAPH_CACHE=off`) : si le résultat change, le défaut est dans la réutilisation, pas dans le résolveur — et la moitié des faux rapports se trie là.

Le guide de contribution impose une définition de fini en cinq points, dont **les preuves collectées, commandes et sorties**, et le passage de `pnpm lint`, `pnpm type-check` et `pnpm test` ; un rapport sans commande ni sortie ne remplit pas le contrat du projet.

Une correction se fait sur une branche courte `task/T-00X-short-name`, une tâche par branche et par pull request.

Trou de couverture n° 2 : aucune politique de sécurité n'est publiée, donc aucun canal privé où signaler une faille — un défaut exploitable n'a d'autre voie que la file publique.

☑ Quelles zones sont peu couvertes ?

VERDICT

Les quatre services ont chacun leur ensemble de tests ; l'outillage qui les livre — scripts, docker, schémas, budgets de performance — n'en a aucun qui porte son nom.

Le code avance par salves : 11 commits relevés sur la seule semaine 36, zéro sur les onze semaines précédentes, dernier commit le 6 septembre 2026 — une campagne de test doit donc viser la fenêtre qui suit une salve, pas un rythme régulier.

Un seul contributeur, `jordanvalnet`, et un bus factor de 1 : aucune relecture croisée humaine n'intervient avant la fusion, la chaîne de vérification est le seul relecteur.

Zones où le relevé ne montre aucun ensemble de tests unitaires portant leur nom, alors que du code y vit :

- ❑ `scripts/` — bootstrap, recettes et benchmark ; vérifiés seulement en s'exécutant dans la chaîne, jamais sur leurs cas d'erreur.
- ❑ `docker/` — composition et profils, aucune recette relevée qui les démarre.
- ❑ `schemas/` — schémas livrés avec le paquet, dont rien ne dit qu'ils sont validés contre les réponses réelles.
- ❑ `perf/` — budgets de performance, isolés dans leur propre workflow, dont les déclencheurs ne sont pas relevés.

La réutilisation du graphe entre deux exécutions est décrite en détail par le projet — empreintes de date et de taille, fenêtre de deux secondes sur les fichiers récemment écrits, invalidation par version — mais elle n'apparaît sous aucun nom d'ensemble de tests dans le relevé : c'est la zone la plus complexe et la moins visiblement couverte.

Donnée non relevée : le taux de couverture par service et la liste des cas de test ; à lire dans le rapport de couverture du dernier passage, <https://github.com/jordanvalnet/code-dev-intel.ts/actions>¹.



Plan de test en 5 points

- 1 **Node 18 et Node 20, sur le paquet publié.** Le manifeste annonce Node 18 ou plus, Node 18 n'est exécuté nulle part et Node 20 ne voit que le binaire compilé : installer l'archive publiée sur ces deux versions et rejouer l'auto-test du serveur. Verdict attendu : soit ça passe, soit `engines` doit être corrigé.
- 2 **Réutilisation du graphe, à froid contre à chaud.** Même dépôt, même question, deux fois : une fois avec `CODE_INTEL_GRAPH_CACHE=off` , une fois sans. Puis les quatre événements de l'édition — fichier ajouté, supprimé, renommé, ressource ajoutée — et une modification de `tsconfig.json` . Toute divergence entre les deux modes est un défaut de premier ordre.
- 3 **Cohérence entre `dependencyGraph` et `impactedFiles` .** Les deux partagent résolveur et cache : sur un même dépôt, le nombre d'éléments non résolus et les arêtes doivent concorder. Vérifier aussi que `unresolvedCount: 0` est bien tenu, et que les quatre causes annoncées — fichier introuvable, type de fichier non lu, hors du répertoire de travail, spécificateur dynamique — sont bien remontées.
- 4 **Les trois surfaces d'entrée, sur les trois systèmes.** `stdio`, `POST /mcp` et `/tools/*` doivent donner la même réponse au même appel ; `ensure` doit rester idempotent avec un port occupé et deux appels concurrents. Windows et macOS sont déjà dans la matrice de vérification, mais seulement pour la suite de tests, pas pour ces parcours.
- 5 **Instrumenter la file avant de commencer.** Créer les étiquettes `bug` , `regression` et `blocked` — aucune n'existe aujourd'hui — et ouvrir un canal de signalement de faille, absent lui aussi ; sinon les défauts trouvés lors des points 1 à 4 ne seront ni comptés, ni suivis.

RÉFÉRENCES PUBLIQUES

☑ Sources

- 1 <https://github.com/jordanvalnet/code-dev-intel.ts/actions>
- 2 <https://github.com/jordanvalnet/code-dev-intel.ts/tree/main/.github/workflows>
- 3 <https://github.com/jordanvalnet/code-dev-intel.ts/issues>